

A Machine-Checked Tree-Graph Inequality and Kotecký–Preiss Cluster-Expansion Engine in Lean 4

Lluis Eriksson

September 2, 2026

Abstract

We describe a complete, `sorry`-free formalization in Lean 4 over Mathlib of the core analytic-combinatorial engine of abstract polymer cluster expansions: the Penrose tree-graph inequality $|\varphi(X)| \leq \#\{\text{spanning trees of the incompatibility graph of } X\}$, proved by a partition scheme built on a greedy breadth-first Penrose map with Boolean-interval fibers; a family of tree-walk (leaf-removal) bounds; the Kotecký–Preiss convergence theorem in both its uniform-majorant form and its sharp, weight-respecting form $\sum_n (\text{pinned cluster weights}) \leq \|z(c)\| e^{a(c)}$ under the bare criterion $\sum_{Y \not\sim X} \|z(Y)\| e^{a(Y)} \leq a(X)$; the Mayer–Ursell inversion $\Xi = \exp(\text{cluster sum})$ for every finite polymer system with absolutely convergent cluster series; and the consequences the programme consumes — restriction to sub-volumes and Z -ratio bounds, activity tilting and exponential size tails, a zero-free activity polydisc, and two lattice-animal counting theorems. The development is a 35-module, 14,441-line layer (`YangMills/KP/`) plus its lattice-side consumers; every theorem stated here depends on exactly Lean’s three standard axioms, `[propxt, Classical.choice, Quot.sound]`, as recorded in the repository’s verification ledger and committed oracle transcripts, and no theorem carries a project axiom. We state each result in ordinary notation next to its Lean declaration, explain the design decisions (finite polymer types, coordinate-pinned clusters, admissible parent/level structures as the tree index, exponentials of `tsums` regrouped through a sigma type, volume-uniformity through per-polymer bounds), tabulate the axiom oracle, and list what is *not* formalized. To our knowledge — after a literature and library search — this is the first machine-checked proof of the Penrose tree-graph inequality and of the Kotecký–Preiss criterion in any proof assistant; the companion papers on the volume-uniform area law and on the thermodynamic limit are its first consumers.

1 Introduction

Cluster expansions are the workhorse of rigorous statistical mechanics in perturbative regimes. Given a family of *polymers* with a hard-core incompatibility relation and small complex *activities*, the polymer partition function Ξ is a finite sum over compatible families, and the cluster expansion expresses $\log \Xi$ as an absolutely convergent series indexed by *clusters*: tuples of polymers whose incompatibility graph is connected, weighted by the Ursell (Mayer) coefficient φ . Three results carry the whole theory. The *tree-graph inequality* of Penrose [2] bounds $|\varphi|$ by a count of spanning trees; the *Kotecký–Preiss criterion* [3] turns that combinatorial bound into absolute convergence with explicit, per-polymer constants; and the *Mayer–Ursell inversion* identifies the sum of the series with $\log \Xi$. Modern expositions — Brydges’ lectures [4], Ueltschi’s review [7], the sharpened criteria of Fernández–Procacci [10] and Bissacot–Fernández–Procacci [11], Faris’ combinatorial account [12], and Chapter 5 of Friedli–Velenik’s textbook [13] — present the material as a small number of clean theorems. The proofs, however, are exactly the kind of mathematics in which

errors hide in bookkeeping rather than in ideas: sign-reversing involutions on edge sets, symmetry factors between ordered tuples and multisets, factorial weights cancelling against tree counts, and the interchange of an exponential with an infinite sum. That makes them a natural target for machine verification, and also a genuine test of whether current proof assistants can absorb a piece of constructive statistical mechanics end to end.

This paper reports such a verification. Within the author’s Lean 4 programme on lattice Yang–Mills theory (repository `THE-ERIKSSON-PROGRAMME`; see [17, 18]), the directory `YangMills/KP/` contains a 35-module, 14,441-line formalization of the abstract polymer cluster expansion, built over Mathlib [16] on Lean 4 [15]. Two earlier papers in the programme are *consumers* of this layer: the volume-uniform Wilson-loop area law [20] (viXra 2607.0005) and the thermodynamic limit of local Gibbs states [21] (viXra 2607.0091). Both describe the engine only in passing. The present paper is the missing account of the engine itself: precise statements, the design decisions that made the formalization tractable, and the axiom oracle.

Contributions.

1. **The Penrose tree-graph inequality, machine-checked** (`abs_ursell_le_card_spanningTrees`, Section 4): for every polymer system and every tuple X , $|\varphi(X)| \leq \#\mathcal{T}(G_X)$. The proof formalizes the partition-scheme mechanism (Boolean-interval fibers telescope) and the concrete greedy breadth-first Penrose scheme, including the fiber characterization that is usually left to the reader.
2. **Tree-walk bounds** (Section 5): an abstract leaf-removal lemma `tree_walk_bound` and its vertexwise and root-pinned variants, which convert per-tree sums of pair kernels into products of one-step budgets.
3. **Kotecký–Preiss convergence in two strengths** (Sections 5–6): the uniform-majorant bound $\text{clusterWeight}_n \leq (\sum \|z\|)(eA)^n$, and the sharp, per-polymer bound $\sum_{n \leq N} \text{pinnedClusterWeight}(c, n) \leq \|z(c)\|e^{a(c)}$ under the bare criterion (`kp_pinned_cluster_bound`). The latter is the form that survives the infinite-volume limit.
4. **The Mayer–Ursell inversion** (`partition_eq_exp_clusterSum`, Section 7): $\Xi = \exp(\text{clusterSum})$ for every finite polymer system whose cluster series is absolutely convergent, hence under the KP criterion; and, as a corollary, non-vanishing of Ξ on a closed activity polydisc (`partition_withActivity_ne_zero_of_kp`).
5. **The volume-uniform toolkit** (Section 8): restriction of a polymer system to a sub-volume with the Z -ratio exponent bound, activity tilting with exponential size tails, and two lattice-animal counting theorems (`card_connectedPolymers_le` for plaquette polymers, `animal_card_le` for bounded-degree graphs) that make the criterion’s smallness window depend only on the dimension.

What is claimed and what is not. Everything stated as a theorem below is a Lean declaration whose axiom oracle prints `[propext, Classical.choice, Quot.sound]` in the repository’s records (Section 10); no theorem carries a `sorry` or a `project` axiom. The mathematics is classical; the contribution is the mechanization and the explicitness. Section 11 lists what is not formalized: infinite polymer systems, the Fernández–Procacci sharpening, the Brydges–Kennedy / Abdesselam–Rivasseau tree identities, the exact Cayley count, and the correlation-function cluster expansion.

2 Related work

Mathematics. Penrose’s partition scheme [2] (see also [1]) is the source of the tree-graph inequality in the form used here; the Boolean-interval formulation follows the modern presentations in [10, 12, 8]. Kotecký and Preiss [3] proved convergence by induction on cluster size under the criterion $\sum_{Y \not\sim X} \|z(Y)\| e^{a(Y)} \leq a(X)$; Dobrushin [5] and Miracle-Solé [6] gave alternative proofs, Ueltschi [7] an efficient one, and Fernández–Procacci [10] and Bissacot–Fernández–Procacci [11] sharper criteria. Brydges’ lectures [4] and Friedli–Velenik [13] (Theorem 5.4, Proposition 5.3) are the expository references whose statements the formalization tracks most closely. The connection between the polymer gas and the Lovász local lemma [9] is relevant to the formalization landscape below.

Formalizations. We searched (September 2026) the Lean Mathlib library, the Isabelle Archive of Formal Proofs, the Coq/Rocq opam ecosystem, and the general literature for a formalization of the tree-graph inequality, the Kotecký–Preiss theorem, the Mayer expansion, or an abstract polymer model. We found none. The repository’s own planning document (`docs/kp-cluster-expansion-plan.md`, May 2026) already recorded that Mathlib contains no polymer model, no Ursell coefficient and no cluster expansion; that remains the case at the pinned Mathlib revision. The closest neighbours are formalizations of the probabilistic method and of the Lovász local lemma (in Isabelle/HOL, Edmonds [14]), which share the combinatorial flavour of the independent-set polynomial but do not touch Ursell coefficients, tree-graph bounds or convergence criteria. We therefore believe — with the usual caveat that absence of evidence in a search is not proof of absence — that the results of Sections 4–7 are the first machine-checked proofs of these theorems in any proof assistant. Within the programme itself, the Catalan/rooted-tree majorant material of [19, 22] is a separate, already documented lane (`YangMills/KP/RootedCatalan*.lean`, `PlaneTreeCatalan.lean`, `LabeledPlaneTreeCatalan.lean`); it is not re-described here.

3 The abstract polymer system in Lean

All definitions live in namespace `YangMills.KP`. We write $\mathcal{P}$ for the polymer type, $X \not\sim Y$ for incompatibility and z for the activity.

Definition 3.1 (polymer system; `YangMills/KP/Basic.lean`). A *polymer system* is a structure

```
structure PolymerSystem where
  Polymer : Type*
  incomp : Polymer → Polymer → Prop
  incomp_symm : ∀ X Y, incomp X Y → incomp Y X
  incomp_self : ∀ X, incomp X X
  activity : Polymer → ℂ
```

A finite family S is *admissible* (`Admissible`) if no two distinct members are incompatible, and the *partition function* of a finite region $\Lambda \subseteq \mathcal{P}$ is

$$\Xi(\Lambda) = \sum_{\substack{S \subseteq \Lambda \\ S \text{ admissible}}} \prod_{X \in S} z(X) \quad (\text{partition}).$$

The hard core is built in as reflexivity of $\not\sim$; this is the Kotecký–Preiss convention $\gamma \not\sim \gamma$ and it is what makes the criterion sum below contain the X -term. The first nontrivial fact is the factorization

$\Xi(\Lambda_1 \cup \Lambda_2) = \Xi(\Lambda_1)\Xi(\Lambda_2)$ over disjoint, mutually compatible regions (`partition_union`), proved by an explicit bijection $S \mapsto (S \cap \Lambda_1, S \cap \Lambda_2)$.

Definition 3.2 (clusters and the Ursell coefficient; `Cluster.lean`, `Ursell.lean`). For an n -tuple $X : \text{Fin } n \rightarrow \mathcal{P}$ the *incompatibility graph* G_X (`incompGraph`) is the simple graph on $\text{Fin } n$ with $i \sim j$ iff $i \neq j$ and $X_i \not\sim X_j$; X is a *cluster* (`IsCluster`) iff G_X is connected. The *Ursell coefficient* is the signed count of connected spanning subgraphs,

$$\varphi(X) = \sum_{\substack{E \subseteq E(G_X) \\ (\text{Fin } n, E) \text{ connected}}} (-1)^{|E|} \in \mathbb{Z} \quad (\text{ursell}).$$

```
noncomputable def ursell {n : ℕ} (X : Fin n → P.Polymer) : ℤ := by
  classical
  exact ∑ E ∈ (incompGraph P X).edgeFinset.powerset.filter
    (fun E : Finset (Sym2 (Fin n)) =>
      (SimpleGraph.fromEdgeSet (↑E : Set (Sym2 (Fin n))))).Connected),
    (-1 : ℤ) ^ E.card
```

Spanning subgraphs are represented as `Finsets` of edges $E \subseteq E(G_X)$, and the graph they span is Mathlib’s `SimpleGraph.fromEdgeSet`. This choice — edge finsets on a fixed vertex type $\text{Fin } n$ rather than Mathlib’s `Subgraph` — is used consistently through the Penrose scheme and the Mayer inversion, and it is the single most consequential representation decision in the layer: Boolean intervals $\{E : T \subseteq E \subseteq R\}$, sign-reversing edge toggles, and component partitions are all `Finset` operations. Sanity checks proved on the definition: $\varphi = 1$ on singletons, -1 on an incompatible pair, $+2$ on a triangle (`ursell_fin_one`, `ursell_fin_two`, `ursell_fin_three`, the last by kernel `decide` over the eight spanning subgraphs), and $\varphi(X) = 0$ whenever X is not a cluster (`ursell_eq_zero_of_not_isCluster`). On the complete graph the recurrence $d(n+1) = -n d(n)$ and the closed form $\varphi(K_{n+1}) = (-1)^n n!$ are theorems (`ursellComplete_recurrence`, `ursellComplete_eq`, `UrsellRecurrence.lean`), proved by fibering the vanishing all-subgraphs sum by the component of vertex 0 and killing the large fibers with a sign-reversing edge-toggle involution.

Definition 3.3 (cluster sum, weights, criterion; `Expansion.lean`, `ClusterWeight.lean`, `PinnedCluster.lean`, `Criterion.lean`). For a *finite* polymer type (`[Fintype P.Polymer]`):

$$\text{clusterSum}(P) = \sum_{n=0}^{\infty} \frac{1}{(n+1)!} \sum_{X : \text{Fin}(n+1) \rightarrow \mathcal{P}} \varphi(X) \prod_i z(X_i) \in \mathbb{C}, \quad (1)$$

$$\text{clusterWeight}(P, n) = \frac{1}{(n+1)!} \sum_{X : \text{Fin}(n+1) \rightarrow \mathcal{P}} |\varphi(X)| \prod_i \|z(X_i)\|, \quad (2)$$

$$\text{pinnedClusterWeight}(P, c, n) = \frac{1}{(n+1)!} \sum_{\substack{X : \text{Fin}(n+1) \rightarrow \mathcal{P} \\ X_0 = c}} |\varphi(X)| \prod_i \|z(X_i)\|. \quad (3)$$

(in Lean: `clusterSum`, `clusterWeight`, `pinnedClusterWeight`). A weight $a : \mathcal{P} \rightarrow \mathbb{R}$ satisfies the *Kotecký–Preiss criterion* (`KPCriterion`) if $a \geq 0$ and

$$\sum_{Y : X \not\sim Y} \|z(Y)\| e^{a(Y)} \leq a(X) \quad \text{for every } X \in \mathcal{P}. \quad (4)$$

Three design decisions are visible here. (i) *Finite polymer types*. The outer sum in (1) is a genuine `tsum` over the cluster size, but every inner sum is a `Finset` sum; there is no infinite polymer set anywhere in the engine. Volume-uniformity is obtained not by working in infinite volume but by proving bounds whose shape does not mention $\#\mathcal{P}$ (Sections 6 and 8). (ii) *Ordered tuples, not multisets*. Clusters are functions $\text{Fin}(n+1) \rightarrow \mathcal{P}$ with the symmetry factor $1/(n+1)!$; this avoids multiset combinatorics entirely and makes the Ursell coefficient relabeling-invariant by a transported graph isomorphism (`ursell_comp_equiv`). (iii) *Coordinate pinning*. A pinned cluster is one with $X_0 = c$, not one containing c ; pinning is then an exact fiberwise decomposition, $\text{clusterWeight}(P, n) = \sum_c \text{pinnedClusterWeight}(P, c, n)$ (`clusterWeight_eq_sum_pinned`), with no overcounting. (Paper-level remark, not formalized: by symmetry the coordinate-pinned sum equals the sum over tuples *containing* c weighted by the fraction of coordinates equal to c , so the two conventions differ only by that fraction.) Two elementary consequences of the criterion are recorded once and reused: $\|z(X)\| \leq a(X)$ (`kp_activity_le`) and the unweighted neighbour bound $\sum_{Y \not\sim X} \|z(Y)\| \leq a(X)$ (`kp_neighbor_sum_le`), both because the X -term is in the sum and $e^a \geq 1$.

4 The Penrose tree-graph inequality

4.1 Statement

Let $\mathcal{T}(G)$ (`spanningTrees`, `PenrosePrep.lean`) be the finset of edge subsets $E \subseteq E(G)$ such that `fromEdgeSet` E is a tree on the full vertex set `Fin n`.

Theorem 4.1 (Penrose tree-graph inequality). [`abs_ursell_le_card_spanningTrees`, `KP/PenroseFiber.lean`] *For every polymer system P , every n and every $X : \text{Fin } n \rightarrow \mathcal{P}$,*

$$|\varphi(X)| \leq \#\mathcal{T}(G_X).$$

```
theorem abs_ursell_le_card_spanningTrees (P : PolymerSystem) {n : ℕ}
  (X : Fin n → P.Polymer) [Fintype (incompGraph P X).edgeSet] :
  |ursell P X| ≤ ((spanningTrees (incompGraph P X)).card : ℤ)
```

Two uniform corollaries follow (same file). With `treeCount`(n) the number of diagonal-free edge sets that are trees on `Fin n` (`treeCount`), $|\varphi(X)| \leq \text{treeCount}(n)$ (`abs_ursell_le_treeCount`), and $\text{treeCount}(m+1) \leq (m+1)^{m+1}$ (`treeCount_le_pow`), hence $|\varphi(X)| \leq (m+1)^{m+1}$ for every $(m+1)$ -tuple (`abs_ursell_le_succ_pow`). The tree count is *not* Cayley's $(m+1)^{m-1}$: the injection used is the greedy-parent map $T \mapsto \text{bfsParent } T$ into $\text{Fin}(m+1) \rightarrow \text{Fin}(m+1)$, which is trivial to verify once one knows that a spanning tree is recovered from its parent function (`penroseTree_of_spanningTree`). The lost factor $(m+1)^2$ is absorbed by the analytic inequality $(n+1)^n \leq e^n n!$ (`succ_pow_le_exp_mul_factorial`, `PenrosePrep.lean`, an induction from $(1 + 1/k)^k \leq e$), which is all the convergence argument needs.

4.2 Proof architecture: partition schemes

The proof is Penrose's, in the Boolean-interval formulation.

Lemma 4.2 (interval signed sum). [`interval_signed_sum`, `PenroseScheme.lean`] *For finsets $T \subseteq R$, $\sum_{T \subseteq E \subseteq R} (-1)^{|E|} = (-1)^{|T|} [R = T]$.*

Lemma 4.3 (abstract partition scheme). `[abs_signedSum_le_of_scheme]` *Let S be a finset of finsets, Tr a finset (the “trees”), and π, R maps of finsets with $\pi(S) \subseteq \text{Tr}$ and, for every $T \in \text{Tr}$ and every E , $(E \in S \wedge \pi E = T) \iff (T \subseteq E \subseteq R T)$. Then $|\sum_{E \in S} (-1)^{|E|}| \leq \#\text{Tr}$.*

The graph-level statement `abs_ursell_le_card_spanningTrees_of_scheme` instantiates S to the connected spanning edge subsets of G_X and Tr to $\mathcal{T}(G_X)$, carrying the scheme (π, R) and its two properties `hmaps/hfiber` as explicit hypotheses. This separation — an abstract telescoping lemma plus a concrete scheme — is what made the development incremental: the abstract half was proved and oracle-checked weeks before the scheme existed, and the scheme’s two obligations were closed one at a time.

4.3 The greedy BFS scheme and its fibers

The concrete scheme (`PenroseBFS.lean`, `PenroseFiber.lean`) is the classical greedy breadth-first construction on the vertex set $\text{Fin}(m+1)$ rooted at 0:

- `bfsLevel E v` is the graph distance from 0 in `fromEdgeSet E` (Mathlib’s `SimpleGraph.dist`);
- `bfsParent E v` is the *least* neighbour of v one level closer to the root (`Finset.min’` over the candidate set);
- $\pi(E) = \text{penroseTree } E = \{ \{ \text{bfsParent } E v, v \} : v \neq 0 \}$;
- $R(T) = \text{penroseEnvelope } H T$ is the set of edges $\{u, v\}$ of the ambient graph H whose endpoints lie on the same T -level, or on adjacent levels with the lower endpoint not smaller than the greedy parent of the upper one (`penroseRel`).

Property `hmaps` is `penroseTree_mem_spanningTrees`: for connected E supported on H , $\pi(E)$ is connected (induction on the level), has exactly m edges (the child is the higher endpoint, so $v \mapsto \{\text{parent } v, v\}$ is injective), hence is a tree by Mathlib’s `isTree_iff_connected_and_card`. Property `hfiber` is the theorem usually stated without proof:

Theorem 4.4 (Penrose fiber characterization). `[penrose_hfiber]` *For a spanning tree T of H and any edge finset E : E is a connected spanning subgraph of H with $\pi(E) = T$ if and only if $T \subseteq E \subseteq R(T)$.*

The proof needs four facts about trees and BFS levels that Mathlib did not supply directly: distances only shrink in a supergraph (`dist_le_dist_of_le`); a function that changes by at most one across edges grows at most by the length along any walk (`level_le_of_walk`); in a tree every path realizes the distance (`isTree_path_length_eq_dist`, via path uniqueness and the geodesic’s bypass); and adjacent tree vertices lie on consecutive levels (`isTree_adj_level`). From these: on the interval $T \subseteq E \subseteq R(T)$ the BFS levels of E equal those of T (`bfsLevel_eq_of_interval`), the greedy parents agree (`bfsParent_eq_of_interval`), and π fixes spanning trees (`penroseTree_of_spanningTree`, by parent uniqueness `isTree_parent_unique`); conversely π preserves levels and parents (`bfsLevel_penroseTree_eq`, `bfsParent_penroseTree_eq`), which places every edge of a connected E in $R(\pi E)$. The $n = 0$ case of Theorem 4.1 is handled separately ($\varphi = 0$ because no graph on an empty vertex type is connected in Mathlib’s sense).

5 Tree-walk bounds and the uniform Kotecký–Preiss bound

The bridge from tree counting to activity sums is the leaf-removal induction. It is stated once, abstractly, over an arbitrary finite vertex type so that removing a leaf is a plain induction on n (function spaces split at a coordinate via `Equiv.funSplitAt`) rather than a `Fin`-reindexing.

Theorem 5.1 (tree-walk bound). `[tree_walk_bound, KP/WalkBound.lean]` *Let V be a finite type with $\#V = n + 1$, $r \in V$, $p : V \rightarrow V$ a parent map and $\text{lev} : V \rightarrow \mathbb{N}$ with $\text{lev}(pv) < \text{lev}(v)$ for $v \neq r$. Let β be a finite type, $I : \beta \rightarrow \beta \rightarrow \mathbb{R}_{\geq 0}$, $w : \beta \rightarrow \mathbb{R}_{\geq 0}$ and $A \geq 0$ with $\sum_y I(x, y) w(y) \leq A$ for all x . Then*

$$\sum_{X:V \rightarrow \beta} \left(\prod_{v \neq r} I(X_{pv}, X_v) w(X_v) \right) w(X_r) \leq A^n \sum_y w(y).$$

```
theorem tree_walk_bound (n : ℕ) :
  ∀ (V : Type) [Fintype V] [DecidableEq V] (β : Type*) [Fintype β]
  (r : V) (p : V → V) (lev : V → ℕ)
  (I : β → β → ℝ) (w : β → ℝ) (A : ℝ),
  Fintype.card V = n + 1 →
  (∀ v, v ≠ r → lev (p v) < lev v) →
  (∀ x y, 0 ≤ I x y) → (∀ y, 0 ≤ w y) → 0 ≤ A →
  (∀ x : β, ∑ y, I x y * w y ≤ A) →
  ∑ X : V → β,
    (∏ v ∈ Finset.univ.filter (fun v => v ≠ r),
      I (X (p v)) (X v) * w (X v)) * w (X r)
  ≤ A ^ n * ∑ y, w y
```

Two variants are proved in the same style. `tree_walk_bound_vertexwise` lets the kernel I_v and the budget A_v depend on the child vertex, concluding $\leq (\prod_{v \neq r} A_v) \sum_y w_{\text{root}}(y)$; it is the form consumed by degree-sensitive leaf summations in the programme’s renormalization-group lane (`YangMills/RG/AppendixFSecondUrsellLeafSummation.lean`). `tree_walk_bound_pinned` (`PinnedWalk.lean`) fixes the root value $X_r = x_0$ and concludes $\leq A^n$ with *no* extensive factor $\sum_y w(y)$. Its proof is a small trick worth recording: rather than re-running the induction, extend β by a marker `none` of weight T that interacts like x_0 and can never sit at a non-root vertex; the free bound gives $T \cdot (\text{pinned sum}) \leq A^n(T + \sum w)$ for every $T \geq 0$, and one large T forces the claim without any limit.

Theorem 5.2 (uniform-majorant KP bound). `[kp_per_size_bound, kp_convergence, kp_norm_clusterSum_le, KPBound.lean]` *If a satisfies the criterion (4) and $a \leq A$ pointwise, then for every n*

$$\text{clusterWeight}(P, n) \leq \left(\sum_y \|z(y)\| \right) (eA)^n,$$

so for $eA < 1$ the cluster series converges absolutely and $\|\text{clusterSum}(P)\| \leq (\sum_y \|z(y)\|)/(1 - eA)$.

The proof composes Theorem 4.1 (expanding $|\varphi(X)|$ over trees T and dominating $\#\{T \subseteq E(G_X)\}$ by an indicator product over the edges of T), the parent-indexed form of a tree product (`prod_tree_eq_prod_parents`: $\prod_{e \in T} f(e) = \prod_{v \neq 0} f(\{\text{parent } v, v\})$), the per-tree assignment bound `tree_assignment_sum_le` obtained from Theorem 5.1 with $I(x, y) = [x \not\sim y]$, $w = \|z\|$ and the budget from `kp_neighbor_sum_le`, and finally $\text{treeCount}(n + 1)/(n + 1)! \leq e^n$. This is the Kotecký–Preiss theorem in the form “smallness $eA < 1$ ”, with A a uniform majorant of the weight. For a lattice gas with $a(c) = t|c|$ the only uniform majorant is $A = t \cdot \#\mathcal{P}$, which is

volume-dependent; the sharp form of the next section removes A altogether. A pinned version with the same defect, `pinnedClusterWeight( $P, c, n$ )` $\leq \|z(c)\| (eA)^n$ (`kp_pinned_per_size_bound`, `PinnedBound.lean`), served as the stepping stone.

6 The sharp Kotecký–Preiss bound

Theorem 6.1 (sharp pinned bound). [`kp_pinned_cluster_bound`, `KP/SharpShell.lean`] *If a satisfies the criterion (4), then for every polymer c and every truncation N ,*

$$\sum_{n=0}^N \text{pinnedClusterWeight}(P, c, n) \leq \|z(c)\| e^{a(c)}.$$

Consequently (`pinned_cluster_summable_sharp`) the pinned series is summable, with sum at most $\|z(c)\|e^{a(c)}$; the total weights are summable (`kp_clusterWeight_summable_sharp`); the cluster series converges absolutely (`kp_convergence_sharp`, `SharpKP.lean`); and $\|\text{clusterSum}(P)\| \leq \sum_c \|z(c)\|e^{a(c)}$ (`kp_norm_clusterSum_le_sharp`).

```
theorem kp_pinned_cluster_bound (P : PolymerSystem) [Fintype P.Polymer]
  {a : P.Polymer → ℝ} (h : KPCriterion P a) (c : P.Polymer) (N : ℕ) :
  ∑ n ∈ Finset.range (N + 1), pinnedClusterWeight P c n
  ≤ ‖P.activity c‖ * Real.exp (a c)
```

No uniform majorant, no geometric smallness hypothesis, and no quantity of the shape $\#\mathcal{P}$ appears: the right-hand side is a per-polymer quantity. This is the statement that Friedli–Velenik’s Theorem 5.4 and Ueltschi’s review deliver; the formalized route is neither the original Kotecký–Preiss induction on cluster size nor Ueltschi’s, but a *shell decomposition* of rooted trees, chosen (in `docs/SHARP-KP-PLAN.md`) because it reuses Theorem 4.1 only through rooted-tree data and keeps every sum finite.

The analytic half (`SharpMajorant.lean`). Define the depth- D majorant by the recursion

$$\Phi_0(c) = 1, \quad \Phi_{D+1}(c) = \exp\left(\sum_{c' : c \not\sim c'} \|z(c')\| \Phi_D(c')\right) \quad (\text{kpMajorant}).$$

Under the criterion, $\Phi_D(c) \leq e^{a(c)}$ for every depth (`kpMajorant_le_exp`): induction on D , the criterion absorbing one shell per step. This is the only place the criterion is used.

The combinatorial half (`SharpShell.lean`, 3,079 lines). An *admissible structure* on $\text{Fin}(n+1)$ is a pair (p, lev) of a parent map and a level map into $\text{Fin}(D+1)$ with $\text{lev}(0) = 0$ and $\text{lev}(pv) + 1 = \text{lev}(v)$ for $v \neq 0$ (`IsAdmissible`). The *raw rooted tree-sum* pinned at c is

$$\text{treeSumRaw}(c, D, n) = \sum_{(p, \text{lev}) \text{ admissible}, p(0)=0} \sum_{X : X_0=c} \prod_{v \neq 0} [X_{pv} \not\sim X_v] \|z(X_v)\|.$$

Step C2 (`pinnedClusterWeight_le_treeSumRaw`) dominates `pinnedClusterWeight( $P, c, n$ )` by $\frac{1}{(n+1)!} \|z(c)\| \text{treeSumRaw}(c, n, n)$: expand $|\varphi|$ over spanning trees by Theorem 4.1, and embed each tree injectively into admissible structures via its BFS parent and level maps (levels fit into

$\text{Fin}(n+1)$ by `connected_dist_lt_card`). The master shell recursion is then a statement about truncated Borel sums $B_D^N(c) = \sum_{n \leq N} \text{treeSumRaw}(c, D, n)/n!$ (`treeSumB`):

$$B_{D+1}^N(c) \leq \exp\left(\sum_{c'} [c \not\sim c'] \|z(c')\| B_D^N(c')\right) \quad (\text{treeSumB_succ_le}),$$

proved by cutting each admissible structure at the first shell (the children of the root), factorizing the assignment sum over the subtrees hanging off the shell (`inner_factorization`, `master_partition`), pricing the enumeration of shells by multinomial coefficients (`card_blockData_mul_le`, `per_k_bound`, `rho_sum_le_price`), and recognizing the exponential series in the $1/k!$ -weighted size-vector sums (`treeSumRaw_succ_le`). Induction on D gives $B_D^N(c) \leq \Phi_D(c) \leq e^{a(c)}$ (`treeSumB_le_kpMajorant`, `treeSumB_le_exp`); depth monotonicity of `treeSumRaw` and the factorial comparison $1/(n+1)! \leq 1/n!$ assemble Theorem 6.1.

Why admissible parent/level pairs rather than trees? Because they are a *decidable, finite, product-shaped* index (a filter on $(\text{Fin}(n+1) \rightarrow \text{Fin}(n+1)) \times (\text{Fin}(n+1) \rightarrow \text{Fin}(D+1))$), over-counting trees harmlessly (the inequality only needs an injection from trees), while making “the first shell” and “the subtree rooted at s ” definable by simple arithmetic on levels (`shellRoot`, `shellFiber`, `subtreeParent`, `subtreeLev`). Nothing in the recursion manipulates a graph.

7 The Mayer–Ursell inversion

Theorem 7.1 (Mayer–Ursell inversion). [`partition_eq_exp_clusterSum`, `KP/MayerInversion.lean`] *Let P be a finite polymer system whose cluster series is absolutely convergent, i.e. $\sum_n \|\frac{1}{(n+1)!} \sum_X \varphi(X) \prod_i z(X_i)\| < \infty$. Then*

$$\Xi(\mathcal{P}) = \exp(\text{clusterSum}(P)).$$

In particular (`partition_eq_exp_clusterSum_of_kp`) this holds under the criterion (4), and then $\Xi(\mathcal{P}) \neq 0$.

```
theorem partition_eq_exp_clusterSum [Fintype P.Polymer]
  (hA : Summable fun n : ℕ => \|((n + 1).factorial : ℂ))-1 *
    ∑ X : Fin (n + 1) → P.Polymer,
    (ursell P X : ℂ) * ∏ i, P.activity (X i)\|) :
  partition P (Finset.univ : Finset P.Polymer)
  = Complex.exp (clusterSum P)
```

This is the fundamental theorem of cluster expansions (Friedli–Velenik Proposition 5.3, in the “ $\Xi = \exp$ ” rather than “ $\log \Xi =$ ” form, which avoids any branch of the complex logarithm). Its formalization has a finite combinatorial heart and an analytic shell.

The partition identity. For a tuple X , call X *pairwise compatible* (`PairwiseCompatible`) if no two distinct coordinates are incompatible. The indicator identity $\sum_{E \subseteq E(G_X)} (-1)^{|E|} = [X \text{ pairwise compatible}]$ (`sum_neg_one_pow_eq_indicator`) is Mathlib’s `Finset.sum_powerset_neg_one_pow_card`. Grouping edge sets by the partition of $\text{Fin } n$ into connected components (`componentPartition`, a `Finpartition` obtained from Mathlib’s reachability setoid) and factorizing each fiber over its blocks (`fiber_cp_factorization`) yields

$$\sum_{\pi \in \text{Finpartition}(\text{Fin } n)} \prod_{B \in \pi} \varphi(X|_B) = [X \text{ pairwise compatible}] \quad (\text{ursell_partition_identity}), \quad (5)$$

where $X|_B$ is X restricted to the block B via Mathlib's order isomorphism $\text{Fin } |B| \simeq B$. Summing (5) against $\prod_j z(X_j)$ over all N -tuples, converting unordered partitions into ordered ones with a $1/k!$ (`sum_finpartition_eq_ordPartitions`, `IsOrdPartition`) and counting the tuples with prescribed block sizes by multinomials (`card_ordPartition_mul`, `sum_ordp_fiber_sizes`), one arrives at the *fully finite cluster form* of Ξ (`partition_univ_eq_cluster_layers`): $\Xi(\mathcal{P})$ equals a finite sum over total sizes $N \leq \#\mathcal{P}$ of $1/k!$ -weighted sums over size vectors $(m_1, \dots, m_k)$, $\sum m_i = N$, $m_i \geq 1$, of products of normalized per-block cluster weights $\frac{1}{m_i!} \sum_{Y: \text{Fin } m_i \rightarrow \mathcal{P}} \varphi(Y) \prod z(Y_l)$. This is where the injectivity of pairwise compatible tuples (`PairwiseCompatible.injective`, from the hard core) turns the tuple sum into the admissible-set sum Ξ .

The exponential of a tsum. Write a_n for the n -th term of (1), so $\text{clusterSum} = \sum' a_n$. Under absolute convergence,

$$\exp\left(\sum_n' a_n\right) = \sum_{(k,f) \in \Sigma_k(\text{Fin } k \rightarrow \mathbb{N})}' \frac{1}{k!} \prod_i a_{f(i)} \quad (\text{exp_tsum_eq_tsum_H}),$$

by Mathlib's power series for $\exp$ (`NormedSpace.exp_eq_tsum_div`), the identity $(\sum' a)^k = \sum_f' \prod_i a_{f(i)}$ (`tsum_pow_eq_tsum_pi`, by induction on k through `tsum_mul_tsum_of_summable_norm`) and `Summable.tsum_sigma`. The master family on the sigma type is absolutely summable by comparison with the exponential series (`summable_H`), and regrouping it by total size $N = \sum_i (f(i) + 1)$ (`tsum_H_eq_tsum_layers`) produces exactly the finite layers of the cluster form, term by term (`layer_shift`). Since the layers vanish for $N > \#\mathcal{P}$, the `tsum` over N is a finite sum and Theorem 7.1 follows. This is how the “restricted” inversion avoids infinite sums on the partition-function side: Ξ is a polynomial in the activities, the only infinite object is `clusterSum`, and absolute convergence is used exactly once, to justify the sigma-type regrouping.

A consistency check. On the single-polymer system with $\|z\| < 1$ the theorem specializes, via $\varphi(K_{n+1}) = (-1)^n n!$ and the Mercator series (`mayer_log_series_complex`, from Mathlib's `Complex.hasSum_taylorSeries_log`), to $\text{clusterSum} = \log(1 + z)$ and $\Xi = 1 + z$ (`clusterSum_singlePolymer_eq_log`, `partition_singlePolymer_eq_exp`, `UrsellRecurrence.lean`). This was proved before the general theorem and served as the normalization check between `partition` and `clusterSum`.

Zero-free polydisc (`ActivityDomain.lean`). The criterion reads z only through $\|z\|$, so it is monotone under pointwise domination (`kpCriterion_withActivity_of_le`), and Theorem 7.1 gives:

Corollary 7.2. [`partition_withActivity_ne_zero_of_kp`, `partition_diskFamily_ne_zero_of_kp`] *If a satisfies (4) for P and z' is any activity with $\|z'(X)\| \leq \|z(X)\|$ for all X , then $\Xi_{z'}(\mathcal{P}) \neq 0$. In particular $\Xi_{wz}(\mathcal{P}) \neq 0$ for all complex w with $\|w\| \leq 1$, and $w \mapsto \Xi_{wz}(\mathcal{P})$ is the explicit polynomial $\sum_S \text{admissible } w^{|S|} \prod_{X \in S} z(X)$ (`partition_diskFamily_eq`).*

8 Restriction, tilting, tails, and animals

The consumers need the engine in finite volume $\Lambda \subseteq \mathcal{P}$ and need estimates whose constants do not see $\#\mathcal{P}$. Four small modules provide this.

Restriction (`Restriction.lean`). $P.\text{restrict } \Lambda$ is the polymer system on the subtype $\{X // X \in \Lambda\}$ with inherited incompatibility and activity. Then $\Xi_P(\Lambda) = \Xi_{P.\text{restrict } \Lambda}(\text{univ})$ (`partition_restrict`), the criterion restricts with the restricted weight (`KPCriterion_restrict`, because the restricted criterion sums are sub-sums), and hence *every* finite-volume partition function is an exponential: $\Xi_P(\Lambda) = \exp(\text{clusterSum}(P.\text{restrict } \Lambda))$ (`partition_eq_exp_clusterSum_restrict`). The ratio of two partition functions is therefore exp of a *difference* of cluster sums, and the difference is exactly the tuple sum over clusters meeting Λ^c (`clusterSum_sub_restrict`), whose norm is bounded by the off-region weight $\sum_n \text{offRegionClusterWeight}(P, \Lambda, n)$ (`norm_clusterSum_sub_restrict_le`).

Theorem 8.1 (volume-free Z -ratio exponent). [`tsum_offRegionClusterWeight_le`] *If $t > 0$ and a satisfies the criterion for the e^t -scaled activities $P.\text{scaleActivity}(e^t)$, then*

$$\sum_n \text{offRegionClusterWeight}(P, \Lambda, n) \leq t^{-1} \sum_{c \notin \Lambda} e^t \|z(c)\| e^{a(c)}.$$

The proof pins each off-region cluster at a coordinate outside Λ (`offRegionClusterWeight_le_pinned`), pays the order factor $n + 1$ by the tilt $e^{t(n+1)} \geq t(n + 1)$ (`orderFactor_pinnedClusterWeight_le_tilt`), and applies Theorem 6.1 to the scaled system. The algebra of the cancellation, $\|\text{Num}/Z\| \leq \sum_i a_i e^{C_i}$ when $Z = e^\zeta$ and $\|\zeta - \zeta_i\| \leq C_i$ (`norm_div_le_pinned_sum_exp`), is stated separately so that the area-law gate can use it verbatim.

Tilting and size tails (`ClusterTail.lean`). $P.\text{tilt } w$ rescales activities by $e^{w(c)}$ without touching φ (`tilt_ursell` is `rfl`). For a size function $\text{sz} : \mathcal{P} \rightarrow \mathbb{N}$ and the pinned weight restricted to clusters of total size $\geq L$ (`pinnedClusterWeightGE`):

Theorem 8.2 (exponential size tail). [`kp_pinned_cluster_tail_bound`, `pinned_cluster_tail_summable`] *If $\varepsilon \geq 0$ and a satisfies the criterion for $P.\text{tilt}(\varepsilon \text{sz})$, then for all c, L, N*

$$\sum_{n \leq N} \text{pinnedClusterWeightGE}(P, \text{sz}, c, L, n) \leq e^{-\varepsilon L} e^{\varepsilon \text{sz}(c)} \|z(c)\| e^{a(c)}.$$

This is the decay mechanism behind the correlator and thermodynamic-limit consumers: clusters connecting two distant regions are large.

Lattice animals, twice. The criterion (4) for a lattice gas needs an entropy bound on connected polymers. Two independent theorems supply it, in different generality.

- `card_connectedPolymers_le` (`YangMills/L1_GibbsMeasure/ConnectedEntropy.lean`): on the periodic d -dimensional lattice of any size N , the number of connected plaquette sets of cardinality $n + 1$ through a fixed plaquette is at most $(16d + 1)^{2n}$ — every such set is the range of a lazy closed walk of length $2n$ (`exists_covering_lazyWalk`), and there are at most $(16d + 1)^{2n}$ such walks because a plaquette touches at most $16d$ others (`card_plaquettesTouching_le`). The constant does not depend on N .
- `animal_card_le` (`YangMills/RG/AnimalTour.lean`): on any simple graph of maximum degree $\leq \Delta$, any family of S -connected vertex sets of size n containing a root has at most $\Delta^{2(n-1)}$ members. The proof peels the farthest vertex (`exists_peel`: a max-distance vertex is never a cut vertex), builds a spanning closed walk of length $2(n - 1)$ by splicing detours (`exists_spanning_closed_walk`, `exists_detour_walk`), and counts walks by `card_walks`.

`length_le_degree_pow` (at most Δ^ℓ walks of length ℓ from a vertex, from Mathlib's recursive `finsetWalkLength`). The cardinal form `rooted_connected_card_le_pow` gives $c_n \leq (\Delta^2)^n$ and feeds the geometric summability `rooted_connected_weight_summable`.

```
theorem animal_card_le [Fintype V] [DecidableRel G.Adj] (A : Finset (Finset V))
  {Δ n : ℕ} (hΔ : ∀ w, G.degree w ≤ Δ)
  (hA : ∀ S ∈ A, r ∈ S ∧ S.card = n ∧ ∀ x ∈ S, ∃ w : G.Walk r x, IsSWalk S w) :
  A.card ≤ Δ ^ (2 * (n - 1))
```

9 How volume-uniformity is achieved, and the consumers

The engine never leaves finite volume; volume-uniformity is a property of the *shape* of the bounds. Concretely, for the connected lattice gas of the strong-coupling expansion — polymers are nonempty connected plaquette sets, incompatibility is edge-support overlap, activities are gauge expectations of $\prod_p (e^{-\beta \text{pe}(\text{hol}_p)} - 1)$ (`connectedLatticePolymerSystem`, `LatticePolymerSystem.lean`) — the chain is:

Theorem 9.1 (volume-uniform criterion). [`connectedLatticePolymerSystem_kpCriterion_volumeUniform`, `ConnectedEntropy.lean`] *Let $|\text{pe}| \leq B$, $t \geq 0$, $x = (e^{|\beta|B} - 1)e^t$, and suppose $(16d + 1)^2 x < 1$ and $16d \cdot x / (1 - (16d + 1)^2 x) \leq t$. Then the weight $a(c) = t|c|$ satisfies the criterion (4) for the connected lattice gas on the lattice of every linear size N .*

The hypotheses mention d , B , β and t only. The proof steers every polymer Y incompatible with c through a plaquette $p \in Y$ touching c — at most $16d$ per plaquette of c (`card_plaquettesTouching_le`), hence at most $16d|c|$ in all — and bounds each per-plaquette sum $\sum_{Y \ni p} x^{|Y|}$ by the geometric series $x/(1 - (16d + 1)^2 x)$ using `card_connectedPolymers_le` (`sum_connectedPolymers_through_le`). Composed with Theorem 6.1 this gives absolute convergence of the cluster series (`connectedLatticeClusterSum_summable_volumeUniform`) and $\|\text{clusterSum}\| \leq \sum_c \|z(c)\| e^{t|c|}$ (`connectedLatticeClusterSum_norm_le_volumeUniform`) with no hypothesis on N ; the earlier uniform-majorant route needed $et\#\{\text{plaquettes}\} < 1$ (`connectedLatticeClusterSum_summable`), which is visible in the code as the hypothesis the sharp version deleted. The tilted criterion (`connectedLatticePolymerSystem_tilt_kpCriterion_volumeUniform`) and the tail theorem give the volume-uniform size tail `connectedLattice_pinned_tail_volumeUniform`.

The three documented consumers are, briefly:

- **Area law** [20] (`YangMills/L1_GibbsMeasure/RestrictedGate.lean`). The region-restricted partition function of a weighted gas is written, by the truncation device $w \mapsto w\mathbf{1}_F$ and `partition_eq_of_activity_eq_zero`, as a full partition function; `partition_eq_exp_clusterSum_of_kp` and `partition_eq_exp_clusterSum_restrict` turn Z and each far factor into exponentials; `norm_clusterSum_sub_restrict_le`, `tsum_offRegionClusterWeight_le` and `norm_div_le_pinned_sum_exp` cancel Z at a cost charged to the loop's perimeter; the pinned dichotomy is resummed by `sum_pinned_dichotomy_le` using the area-extraction split `sum_ite_pow_le`. The headline `normalized_exp_wilson_loop_area_law` is volume-uniform because every constant it inherits from the engine has the per-polymer shape above.
- **Exponential clustering** (`TwoPlaquetteCorrelator.lean`). `sun_two_plaquette_correlator_bound` bounds the normalized truncated two-plaquette correlation of $SU(N_c)$

lattice gauge theory by $e^{-\epsilon k}$ for plaquettes at touching-graph distance $\geq 2k$, with constants in $d, N_c, \beta, s, t, \epsilon$ only, through the size-tail theorem applied to the marked gas.

- **Thermodynamic limit** [21]. The Cauchy modulus of the finite-volume local expectations is the volume-uniform pinned tail (`ThermodynamicLimit.lean`); a genuine free-boundary exhaustion is compared with the periodic sequence through the same tails (`FreeBoundaryThermodynamicLimit.lean`); and the endpoint theorem `tendsto_integerFreeBoundaryThermodynamicExpectation` (`IntegerThermodynamicLimit.lean`) converges the free-boundary exhaustion to the periodic infinite-volume state.

These papers document their own statements and windows; we list them only to make precise which engine theorems they consume.

10 The axiom oracle

The repository’s verification discipline is a per-theorem `#print axioms`. Table 1 lists every declaration cited above that we treat as a headline, its file, and where its verbatim oracle output is recorded. Three record types occur: the verification ledger `docs/VERIFICATION-LEDGER.md` (verbatim batch outputs at dated checkpoints; the KP/Penrose release audit of 2026-06-10 at 8,209 jobs and its Addenda 5–7 and 59 cover most of the table), the committed transcripts `docs/oracle-transcripts/ORACLE-2026072*.txt` (about 4,600 lines each, one entry per headline of `oracle_check.lean` at that date), and `oracle_check.lean` itself. Every recorded output is `[propxt, Classical.choice, Quot.sound]`.

The revision of `oracle_check.lean` accompanying this paper (branch `paper/kp-engine`) adds a section that prints the axioms of forty-two further declarations from the table — among them `abs_ursell_le_card_spanningTrees`, `penrose_hfiber`, `tree_walk_bound`, `kp_pinned_cluster_bound`, `kp_convergence_sharp`, `ursell_partition_identity` and the volume-uniform lattice criterion — which were previously recorded only in the ledger. For full disclosure: the additions were made by static inspection of the sources (every name was checked to exist at the cited file and line) and have *not* yet been re-run through the compiler on this machine; their oracle outputs in the table are those recorded in the ledger at earlier checkpoints, and the continuous-integration workflow does not compile Lean (it enforces the `zero-sorry`, `zero-axiom` source invariants; see Section 12). Re-running `lake env lean oracle_check.lean` at the current checkpoint is the one-line check that closes this gap.

Table 1: Headline declarations, files (under `YangMills/`; `L1` abbreviates `L1_GibbsMeasure`), and oracle records. “L” = verification ledger (verbatim), “T” = committed oracle transcripts of 2026-07-13/27, “O” = present in `oracle_check.lean` before this revision, “O⁺” = added on branch `paper/kp-engine`, “R” = batch statement in `REPRODUCIBILITY.md` (merged oracle script run to exit 0 on `main` checkpoint `f0720ba7`; no per-declaration line quoted). Every recorded output is `[propxt, Classical.choice, Quot.sound]`.

Declaration (<code>YangMills.</code> prefix omitted)	File	Record
<i>Substrate</i>		
<code>KP.partition_union</code>	<code>KP/Basic.lean</code>	O ⁺
<code>KP.ursell_eq_zero_of_not_isCluster</code>	<code>KP/Ursell.lean</code>	O ⁺
<code>KP.ursellComplete_recurrence</code>	<code>KP/UrsellRecurrence.lean</code>	L (2026-06-10), O ⁺
<code>KP.ursellComplete_eq</code>	<code>KP/UrsellRecurrence.lean</code>	L, O ⁺

Declaration	File	Record
KP.partition_singlePolymer_eq_exp	KP/UrsellRecurrence.lean	L, O ⁺
KP.kp_activity_le, KP.kp_neighbor_sum_le	KP/Criterion.lean	O ⁺
<i>Penrose tree-graph inequality</i>		
KP.interval_signed_sum	KP/PenroseScheme.lean	L, O ⁺
KP.abs_signedSum_le_of_scheme	KP/PenroseScheme.lean	L, O ⁺
KP.penroseTree_mem_spanningTrees	KP/PenroseBFS.lean	L, O ⁺
KP.penrose_hfiber	KP/PenroseFiber.lean	L, O ⁺
KP.abs_ursell_le_card_spanningTrees	KP/PenroseFiber.lean	L, O ⁺
KP.abs_ursell_le_treeCount, KP.treeCount_le_pow	KP/PenroseFiber.lean	L, O ⁺
KP.succ_pow_le_exp_mul_factorial	KP/PenrosePrep.lean	L, O ⁺
<i>Tree-walk bounds and uniform KP</i>		
KP.tree_walk_bound	KP/WalkBound.lean	L, O ⁺
KP.tree_walk_bound_vertexwise	KP/WalkBound.lean	T, O
KP.tree_walk_bound_pinned	KP/PinnedWalk.lean	O ⁺
KP.kp_per_size_bound, KP.kp_convergence, KP.kp_norm_clusterSum_le	KP/KPBound.lean	L, O ⁺
<i>Sharp KP</i>		
KP.clusterWeight_eq_sum_pinned	KP/PinnedCluster.lean	O ⁺
KP.kpMajorant_le_exp	KP/SharpMajorant.lean	O ⁺
KP.pinnedClusterWeight_le_treeSumRaw	KP/SharpShell.lean	O ⁺
KP.treeSumB_succ_le, KP.treeSumB_le_kpMajorant	KP/SharpShell.lean	L (Add. 6), O ⁺
KP.kp_pinned_cluster_bound	KP/SharpShell.lean	L (Add. 6), O ⁺
KP.pinned_cluster_summable_sharp	KP/SharpShell.lean	L (Add. 6), O ⁺
KP.kp_clusterWeight_summable_sharp, KP.kp_convergence_sharp, KP.kp_norm_clusterSum_le_sharp	KP/SharpKP.lean	L (Add. 6), O ⁺
<i>Mayer-Ursell inversion and activity domain</i>		
KP.ursell_partition_identity	KP/MayerInversion.lean	L (Add. 7), O ⁺
KP.partition_eq_exp_clusterSum	KP/MayerInversion.lean	L (Add. 7), T, O
KP.partition_eq_exp_clusterSum_of_kp	KP/MayerInversion.lean	L (Add. 7), T, O
KP.kpCriterion_withActivity_of_le	KP/ActivityDomain.lean	T, O
KP.partition_withActivity_ne_zero_of_kp	KP/ActivityDomain.lean	T, O
KP.partition_diskFamily_ne_zero_of_kp, KP.partition_diskFamily_eq	KP/ActivityDomain.lean	T, O
<i>Restriction, tails</i>		
KP.KPCriterion.restrict, KP.partition_eq_exp_clusterSum_restrict	KP/Restriction.lean	L (8,237 jobs), O ⁺
KP.norm_clusterSum_sub_restrict_le, KP.tsum_offRegionClusterWeight_le	KP/Restriction.lean	L (8,237 jobs), O ⁺
KP.orderFactor_pinnedClusterWeight_le_tilt, KP.tsum_finset_pinnedClusterWeight_le	KP/Restriction.lean	T, O
KP.kp_pinned_cluster_tail_bound, KP.pinned_cluster_tail_summable	KP/ClusterTail.lean	O ⁺
<i>Lattice animals and the volume-uniform criterion</i>		
card_connectedPolymers_le	L1/ConnectedEntropy.lean	L (Add. 5), O ⁺
connectedLatticePolymerSystem_kpCriterion_volumeUniform	L1/ConnectedEntropy.lean	L (Add. 5), O ⁺
connectedLatticeClusterSum_summable_volumeUniform	L1/ConnectedEntropy.lean	L (Add. 6), O ⁺
connectedLatticeClusterSum_norm_le_volumeUniform	L1/ConnectedEntropy.lean	L (Add. 6), O ⁺

Declaration	File	Record
<code>connectedLattice_pinned_tail_volumeUniform</code>	<code>L1/ConnectedEntropy.lean</code>	O^+
<code>RG.card_walks_length_le_degree_pow</code>	<code>RG/AnimalCount.lean</code>	T, O
<code>RG.animal_card_le</code>	<code>RG/AnimalTour.lean</code>	L (Add. 59), T, O
<code>RG.rooted_connected_card_le_pow</code>	<code>RG/AnimalTour.lean</code>	T, O
<i>Consumers (documented elsewhere)</i>		
<code>normalized_exp_wilson_loop_areaLaw</code>	<code>L1/RestrictedGate.lean</code>	L, T, O
<code>sun_two_plaquette_correlator_bound</code>	<code>L1/TwoPlaquetteCorrelator.lean</code>	T, O
<code>tendsto_integerFreeBoundaryThermodynamicExpectation</code>	<code>L1/IntegerThermodynamicLimit.lean</code>	R, O

For the declarations marked only O^+ (eleven of them: `partition_union`, `ursell_eq_zero_of_not_isCluster`, `kp_activity_le`, `kp_neighbor_sum_le`, `tree_walk_bound_pinned`, `clusterWeight_eq_sum_pinned`, `kpMajorant_le_exp`, `pinnedClusterWeight_le_treeSumRaw`, `kp_pinned_cluster_tail_bound`, `pinned_cluster_tail_summable` and `connectedLattice_pinned_tail_volumeUniform`) we have no verbatim per-declaration transcript; what is recorded for them is that they compile inside the sorry-free, axiom-free `YangMillsCore` build (latest recorded green build: 8,463 jobs), and they are consumed by declarations whose oracle *is* recorded (a declaration depending on an extra axiom would propagate it to every consumer). We state this rather than assert a transcript we do not have.

11 What is not formalized

In the author’s usual convention, the following are explicitly *not* claimed.

- **Infinite polymer systems.** Every theorem carries `[Fintype P.Polymer]`. The infinite-volume statements of [3, 13] for countable polymer sets are not formalized; volume-uniformity is achieved by the shape of finite-volume bounds and by the consumers’ Cauchy arguments, not by an infinite-volume cluster expansion.
- **The sharp criteria of Fernández–Procacci and Bissacot–Fernández–Procacci** [10, 11], and the Dobrushin form of the criterion. Only the Kotecký–Preiss form (4) is proved.
- **Tree-graph identities** (Brydges–Kennedy, Brydges–Federbush, Abdesselam–Rivasseau) and the Penrose *identity* (as opposed to inequality). The formalized statement is the inequality $|\varphi| \leq \#\mathcal{T}$, which suffices for convergence.
- **Cayley’s formula.** The tree count used is $\text{treeCount}(m+1) \leq (m+1)^{m+1}$, not $(m+1)^{m-1}$; the exact child-factorial Catalan identity over spanning trees of the complete graph is a separate, documented result of the programme [19] and is not used by the engine.
- **The correlation-function cluster expansion** (the cluster representation of truncated correlations, Ueltschi’s Theorem 2). The consumers obtain correlation decay through marked gases and the size-tail theorem instead.
- **The log form.** The inversion is stated as $\Xi = \exp(\text{clusterSum})$; no statement selects a branch of $\log \Xi$.

- **Analytic continuation in the fugacity.** The zero-free polydisc is proved; that $w \mapsto \Xi(wz)$ is entire and that $\log \Xi$ is analytic on the polydisc are on the frontier of the satellite `lean-zero-free-regions`, not in this layer.
- **Cluster expansions with holes / renormalization-group inputs.** The RG lane (`YangMills/RG/`) consumes the vertexwise walk bound and the animal count, but its Balaban–Dimock source estimates are carried as explicit theorem hypotheses (`HYPOTHESIS_FRONTIER.md`) and are not claimed here.
- **Nothing about the continuum.** As every status document of the repository states, no continuum limit, no Osterwalder–Schrader reconstruction and no continuum mass gap is formalized or claimed.

12 Reproduction

All artifacts are in the public repository

<https://github.com/lluiseriksson/THE-ERIKSSON-PROGRAMME>

The engine is `YangMills/KP/` (35 modules); its lattice-side consumers are under `YangMills/L1_GibbsMeasure/`; the graph-theoretic animal count is `YangMills/RG/AnimalCount.lean` and `YangMills/RG/AnimalTour.lean`. The toolchain is pinned: Lean v4.29.0-rc6 (`lean-toolchain`) and the Mathlib commit

07642720480157414db592fa85b626dafb71355b

(`lakefile.lean` and `lake-manifest.json` agree; see `REPRODUCIBILITY.md`). To rebuild the verified core and print the oracle:

```
lake exe cache get           # prebuilt Mathlib oleans
lake build YangMillsCore     # latest recorded: 8463 jobs, green
lake env lean oracle_check.lean # prints the axioms of every headline
python scripts/check_consistency.py # zero sorry, zero core axioms
```

A full core build exceeds the free continuous-integration tier, so the repository’s CI workflow (`.github/workflows/ci.yml`) does *not* compile Lean; it runs the comment-aware scanner `scripts/check_consistency.py`, which fails on any `sorry` in the proof tree or any `axiom` declaration in the verified-core tree. The compiler builds and oracle runs are performed locally and recorded verbatim in `docs/VERIFICATION-LEDGER.md` and `docs/oracle-transcripts/`. This paper’s sources are `papers/kp-engine/main.tex`; the branch `paper/kp-engine` carries the paper, the `oracle_check.lean` additions of Section 10, and (only on that branch) a one-line change to the CI trigger so that the scanner also runs on `paper/**` branches. The document compiles with `latexmk -pdf -interaction=nonstopmode main.tex`; the script `check_listings_unicode.py` audits the Lean listings against the `literate` table.

13 Conclusion

The Penrose tree-graph inequality, the Kotecký–Preiss criterion in its sharp per-polymer form, and the Mayer–Ursell inversion are now theorems of Lean 4 over Mathlib, with the standard three-axiom oracle and no `sorry`. The formalization is faithful to the textbook mathematics but not to its

usual presentation: ordered tuples replace multisets, coordinate pinning replaces membership pinning, admissible parent/level pairs replace trees inside the shell recursion, and the exponential of the cluster series is handled by a sigma-type regrouping rather than by log. Each of these choices traded a conceptual step for a bookkeeping step that Mathlib’s `Finset` and `tsum` libraries could discharge, and together they kept a 14,441-line development oracle-clean at every checkpoint. The layer is already load-bearing: the volume-uniform area law and the thermodynamic limit of the programme are its consumers. Natural next steps are the infinite-polymer version of the criterion, the Fernández–Procacci sharpening, and extraction of the abstract polymer model, the Ursell coefficient and Theorems 4.1 and 7.1 into Mathlib-style generic contributions.

Acknowledgments

The Lean formalization was developed interactively with Anthropic’s Claude, in accordance with the ACM authorship policy on AI-assisted work; every theorem stated here was checked by the Lean 4 kernel, with the axiom oracle outputs archived in the repository ledger and transcripts.

References

- [1] O. Penrose, *Convergence of fugacity expansions for fluids and lattice gases*, J. Math. Phys. **4** (1963), 1312–1320.
- [2] O. Penrose, *Convergence of fugacity expansions for classical systems*, in: T. A. Bak (ed.), *Statistical Mechanics: Foundations and Applications*, Benjamin, New York, 1967, 101–109.
- [3] R. Kotecký and D. Preiss, *Cluster expansion for abstract polymer models*, Comm. Math. Phys. **103** (1986), 491–498.
- [4] D. C. Brydges, *A short course on cluster expansions*, in: K. Osterwalder and R. Stora (eds.), *Critical Phenomena, Random Systems, Gauge Theories* (Les Houches 1984), North-Holland, 1986, 129–183.
- [5] R. L. Dobrushin, *Estimates of semi-invariants for the Ising model at low temperatures*, in: *Topics in Statistical and Theoretical Physics*, Amer. Math. Soc. Transl. (2) **177** (1996), 59–81.
- [6] S. Miracle-Solé, *On the convergence of cluster expansions*, Physica A **279** (2000), 244–249.
- [7] D. Ueltschi, *Cluster expansions and correlation functions*, Moscow Math. J. **4** (2004), 511–522.
- [8] A. D. Scott and A. D. Sokal, *The repulsive lattice gas, the independent-set polynomial, and the Lovász local lemma*, J. Stat. Phys. **118** (2005), 1151–1261.
- [9] A. D. Scott and A. D. Sokal, *On dependency graphs and the lattice gas*, Combin. Probab. Comput. **15** (2006), 253–279.
- [10] R. Fernández and A. Procacci, *Cluster expansion for abstract polymer models. New bounds from an old approach*, Comm. Math. Phys. **274** (2007), 123–140.
- [11] R. Bissacot, R. Fernández and A. Procacci, *On the convergence of cluster expansions for polymer gases*, J. Stat. Phys. **139** (2010), 598–617.
- [12] W. G. Faris, *Combinatorics and cluster expansions*, Probab. Surv. **7** (2010), 157–206.

- [13] S. Friedli and Y. Velenik, *Statistical Mechanics of Lattice Systems: A Concrete Mathematical Introduction*, Cambridge University Press, 2017 (Chapter 5).
- [14] C. Edmonds, *Formal probabilistic methods for combinatorial structures using the Lovász local lemma*, in: CPP 2024, ACM, 2024; arXiv:2310.00513.
- [15] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, in: Automated Deduction – CADE 28, LNCS 12699, Springer, 2021, 625–635.
- [16] The mathlib Community, *The Lean mathematical library*, in: CPP 2020, ACM, 2020, 367–381.
- [17] L. Eriksson, *The Master Map: An Audit-First, Attack-Resistant Navigation Guide to the Unconditional Solution of the 4D $SU(N)$ Yang–Mills Existence and Mass Gap Problem*, ai.viXra:2602.0096, 2026.
- [18] L. Eriksson, *Mechanical Audit Experiments and Reproducibility Appendix for a Companion-Paper Programme on 4D $SU(N)$ Yang–Mills Existence and Mass Gap*, ai.viXra:2602.0117, 2026.
- [19] L. Eriksson, *A Machine-Verified Bijective Proof of the Rooted Child-Factorial Catalan Identity over Spanning Trees of the Complete Graph*, ai.viXra:2607.0001, 2026.
- [20] L. Eriksson, *A Machine-Checked Volume-Uniform Wilson-Loop Area Law via a Formalized Cluster Expansion*, ai.viXra:2607.0005, 2026; repository [paper/area-law/](#).
- [21] L. Eriksson, *A Machine-Checked Thermodynamic Limit for Local Lattice Gauge Gibbs States*, ai.viXra:2607.0091, 2026; repository [papers/thermodynamic-limit-kp/](#).
- [22] L. Eriksson, *Rooted-tree majorants for cluster expansions with holes (C1 paper)*, repository [papers/c1-rooted-tree-majorants/](#), 2026.